

Accent and Language Identification from Speech: A Systematic Comparison of Gradient Boosting and 1D Convolutional Neural Networks on Summary-Statistics and Time-Series MFCC Representations

Yunus Gökdemir
Student ID: 040220060
Istanbul Technical University
gokdemiry22@itu.edu.tr

Delfin Özdemir
Student ID: 040210087
Istanbul Technical University
ozdemird21@itu.edu.tr

Abstract—With the increasing prevalence of speech recognition technologies and the growing importance of speaker profiling, computer-assisted language learning, and automatic speech recognition, the classification of short audio segments in terms of accent and language, which forms the basis of these developments, continues to be crucial. Therefore, examining how to improve model performance in classifying short audio segments is vital. In this context, our study presents a controlled, three-stage comparison that independently tests whether the data representation format presented to the system or the chosen learning algorithm has a greater impact on success. Using 1000 audio recordings for each class, totaling 3000 recordings from the Mozilla Common Voice dataset, two parallel feature groups were created. The first is a structure that calculates and summarizes the statistical properties of the recordings, such as MFCC, zero-crossing rate (ZCR), spectral rolloff, and chroma; the second is a representation that preserves the time-axis variation of the MFCC coefficients. When comparing XGBoost and 1D-CNN models trained with these two representations and having the exact same layer structure, the 1D-CNN model trained with summary features (86.50%) did not achieve a higher accuracy rate than the XGBoost model using gradient boosting (89.00%). In contrast, the time series-based CNN using the second type of feature structure achieved an accuracy rate of 93.67%, providing an improvement of 4.67 points compared to the strongest baseline. As a result, it was observed that the performance improvement was not solely due to the difference in model type, but rather to which features of the dataset were used in training. Within the scope of the study, an interface was created for all three models to enable simultaneous inference, and the system was made available as a web demo.

Index Terms—accent identification, spoken language identification, convolutional neural networks, MFCC, XGBoost, Common Voice

I. INTRODUCTION AND RELATED WORK

One of the fundamental problems that speech processing technologies have been trying to solve for many years is being able to determine a person’s spoken language, or more precisely, their accent, from just a few seconds of audio data. Consistently making this determination enables many innova-

tions, such as accent-adapting speech recognition systems, intelligent routing in call centers, forensic speaker analysis, and personalized feedback in computer-assisted language learning.

The traditional method used to determine spoken language and accent, which also forms the basis of this study, relies on Mel-Frequency Cepstral Coefficients (MFCC) [1] combined with statistical classifiers. The i-vector approach [2], which reduces each audio recording to a single fixed-size vector, has dominated this field for a decade. Voices represented by their statistical properties such as standard deviation and mean can yield good results in algorithms such as XGBoost [3]. Today, deep learning architectures that work directly on frame sequences, such as the x-vector architecture [4] based on time-delayed neural networks, have shown that preserving the temporal integrity of the voice significantly increases success.

This project not only replaces the classic XGBoost method with a 1D-CNN, but also investigates the efficiency of methods that preserve the temporal domain while retaining the statistical properties of the sound. Therefore, a three-stage experiment was created, in which both the model and the data representation were changed sequentially, allowing for the precise and undeniable identification of the primary source of performance improvement. The main contributions of this work are as follows:

- In our work using Google Colab, we created a pipeline that decoded and processed 3,000 Common Voice recordings without saving the audio data to disk, thus keeping the disk footprint negligible while enabling efficient large-scale processing. Thanks to this pipeline, we were able to simultaneously generate both the statistical properties of the data and the time-series MFCC data in a single processing pass.
- The convolutional framework of our proposed compact time-series 1D-CNN model is intentionally designed to be equivalent to a summary-feature CNN; thus, the impact of the input representation format on performance is made

completely independent of the direct effects of the model architecture.

- On a common, stratified test set, the summary-feature CNN was found to perform below the XGBoost model (86.50% compared to 89.00%), but when the time-series representation was used, it performed better than both (93.67%). These results indicate that the factor affecting performance is the temporal structure of the sound rather than the model itself.
- Finally, we make the entire system available as an interactive web demo that performs real-time inference using any of the three models on audio recorded by the user.

II. SYSTEM MODEL AND PROBLEM DEFINITION

A. Problem Definition

Let $x \in \mathbb{R}^{T_x}$ denote a raw speech waveform of variable duration. The task is to learn a classifier

$$f : x \mapsto y \in \{\text{ARABIC, ENGLISH, FRENCH}\}, \quad (1)$$

that predicts the language of the utterance. We study how the choice of the intermediate representation $\phi(x)$ affects the achievable accuracy of f under a fixed data budget.

B. Dataset

In our project, we use the Mozilla Common Voice corpus [5] from the Mozilla Data Collective platform. The first 1,000 MP3 recordings for each of the Arabic, English, and French languages are processed using a pipeline approach that extracts features in the stream without downloading the data to disk. To overcome the problem of limited disk capacity, the tar.gz archive is scanned member by member over HTTP, and each file is parsed in real time, thus reducing the load on the disk. To ensure standard time-series frame rates in the data, all audio clips are resampled to a common rate of 22.05 kHz. The length of the clips is approximately 5 seconds. The resulting 3,000 labeled audio data points are separated into 80% training and 20% test data using stratified sampling techniques; all algorithms are evaluated on the exact same test data points to ensure a completely fair comparison between models.

C. Input Representations

Every recording is mapped to two representations extracted in a single pass:

1) *Summary-statistics vector* (ϕ_{sum}): A 54-dimensional vector consisting of the per-coefficient mean and standard deviation of 13 MFCCs (26 values), the mean and standard deviation of the zero-crossing rate (2), the mean spectral rolloff (1), the per-bin mean and standard deviation of the 12-dimensional chroma vector (24), and the mean RMS energy (1). This representation discards all temporal ordering.

2) *Time-series MFCC matrix* (ϕ_{seq}): The frame-level MFCC matrix $S \in \mathbb{R}^{216 \times 13}$, computed with a hop length of 512 samples (≈ 23 ms per frame, ≈ 5 s total). Shorter clips are zero-padded and longer clips are truncated to exactly 216 frames. This representation preserves the temporal evolution of the spectral envelope, in which accent-specific phonetic patterns reside.

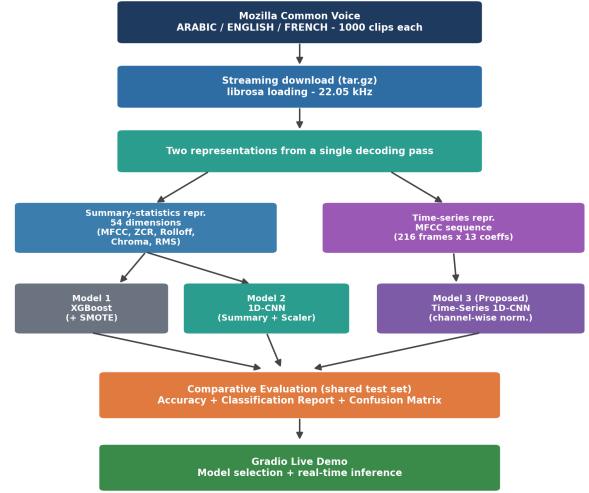


Fig. 1. Overall flowchart of the proposed system. Both representations are extracted from a single decoding pass; all models are evaluated on the identical stratified test set.

III. PROPOSED APPROACH

Fig. 1 gives an overview of the complete system. The three models are described below; their preprocessing steps differ by design, since each step is chosen to be appropriate for the corresponding model family.

A. Model 1: XGBoost Baseline

Our first model, XGBoost, is a gradient-boosting tree ensemble [3] trained on the ϕ_{sum} data, which holds the statistical properties of the sounds. It works by successively adding many small decision trees, where each new tree corrects the errors of the previous one. It has a learning rate of 0.05 and a maximum depth level of 6. The low learning rate and high number of trees prevent the model from memorizing the data and thus avoid overfitting. Since the number of clips in the dataset was set at 1,000 for each language, and some clips were corrupted in different languages, creating an imbalance with different data counts in each language, synthetic data was generated using the SMOTE [6] algorithm for the under-represented classes. To avoid data leakage, this algorithm was applied only to the training set; the data generated with this algorithm was not shown to the model in the test set.

B. Model 2: 1D-CNN on Summary Features

Our second model, 1D-CNN, was trained on the same dataset as the first model, XGBoost, for model comparison purposes. The 54-dimensional vector containing the statistical characteristics of the voice was standardized using `StandardScaler` to eliminate the negative effects of the differences between the scales before being converted to the

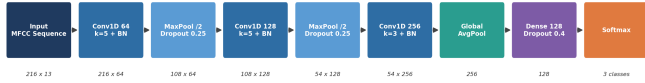


Fig. 2. Architecture of the proposed time-series 1D-CNN (Model 3). Convolutional filters slide along the time axis; the 13 MFCC coefficients act as input channels. Output shapes are given below each block.

matrix format $(54, 1)$. To prevent any leakage at this stage, the operations were applied only to the training set. The model consists of three convolution blocks with increasing numbers of filters (64, 128, 256). These blocks attempt to detect patterns in the data through convolution. Simple patterns are detected in the 64-filter layer, while the layers with 128 and 256 filters capture more complex patterns. A batch normalization [7] step was added to the end of each block to stabilize the data values during training; this step was further supported by max pooling in the first two blocks to reduce the size of the data. Instead of flattening, global average pooling [8] was applied by averaging each filter to reduce the total number of parameters in the network and prevent overfitting caused by an excessive number of parameters. The model was terminated with a fully connected (dense) layer of 128 neurons with a dropout of 0.4 [9] and a softmax output. The main purpose of this model is to answer the critical question: *what does a neural network achieve on exactly the same input as the baseline?*

C. Model 3: Time-Series 1D-CNN (Proposed)

Our latest model accepts as input a matrix that tracks how the audio changes over time, in the format $\phi_{\text{seq}} \in \mathbb{R}^{216 \times 13}$, which was standardized on a channel basis using only the training data to prevent data leakage. The model’s infrastructure has been deliberately kept identical to Model 2 (same filter progression, batch normalization, global average pooling, and fully connected layer). The aim is to prove that any performance difference between the two models stems from the input data format, not the architecture. The most crucial difference is that the 1D convolution filters this time capture patterns in the temporal flow using the 13 MFCC coefficients as channels. In this way, the model preserves and trains on local phonetic features such as vowel linking, speech rhythm, and melody—essentially the key elements determining accent—which were flattened and destroyed by the summary statistics extracted from the audio data used in the first two models. To prevent overfitting due to the increasing data size, a 0.25 dropout has been added to the end of each pooling step. The SMOTE algorithm is not applicable here because generating artificial data (interpolation) between time series would produce acoustically meaningless samples; the class distribution has been designed to be balanced from the outset. The entire structure is schematically represented in Fig. 2.

TABLE I
PER-CLASS AND OVERALL TEST PERFORMANCE (SHARED TEST SET, $n = 600$)

Model	Class	Precision	Recall	F1
XGBoost (Model 1)	Arabic	0.90	0.95	0.92
	English	0.87	0.87	0.87
	French	0.90	0.85	0.88
	Accuracy	89.00%		
Summary 1D-CNN (Model 2)	Arabic	0.86	0.93	0.89
	English	0.84	0.84	0.84
	French	0.90	0.82	0.86
	Accuracy	86.50%		
Time-Series 1D-CNN (Model 3, proposed)	Arabic	0.95	0.97	0.96
	English	0.92	0.91	0.92
	French	0.94	0.93	0.93
	Accuracy	93.67%		

D. Training Protocol

Both network models were trained with an Adam optimizer [10] with a batch size of 32 and an initial learning rate of 10^{-3} to prevent large model jumps, and a sparse categorical cross-entropy function as the loss function. 15% of the training data was reserved for model validation. An early stopping mechanism [11], which prevents overfitting, observes the change in validation loss over 15 epochs and records the best weights. A concurrently running learning rate scheduler halves the learning rate (to a minimum of 10^{-5}) if no improvement is observed over 7 epochs. The models were trained for a maximum of 150 epochs. To ensure the reproducibility of the results, all experiments were run with a constant random seed. TensorFlow/Keras [12] and librosa [13] were chosen as the software infrastructure.

IV. NUMERICAL RESULTS

A. Overall Performance

Table I reports per-class precision, recall, and F1-scores together with the overall accuracy of the three models on the shared test set of 600 utterances (200 per class); Fig. 3 summarizes the accuracies.

Three observations stand out. First, the summary-feature CNN (86.50%) performs worse than XGBoost (89.00%) on identical input: with only 54 aggregate statistics and roughly 2,400 training samples, the inductive bias of gradient-boosted trees is better suited to tabular data than convolutions are. Second, moving the same convolutional backbone to the time-series representation lifts accuracy to 93.67%, improving every per-class F1-score by 4–7 points over the baseline. Since architecture, optimizer, and data split are held fixed between Models 2 and 3, this gain is attributable solely to the preservation of temporal structure. Third, the proposed model is also the most balanced one: its per-class F1-scores span only 0.92–0.96, compared with 0.84–0.89 for Model 2.

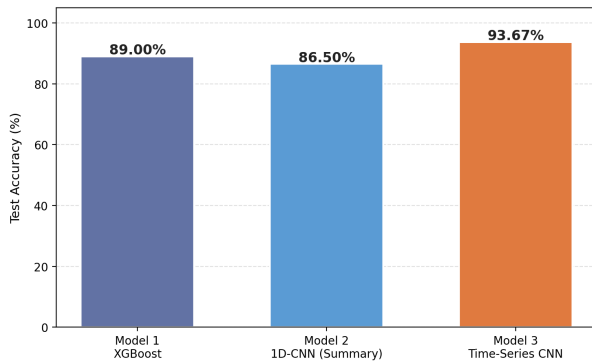


Fig. 3. Test accuracy of the three models. The summary-feature CNN does not surpass the XGBoost baseline; the time-series CNN improves on the baseline by 4.67 points.

B. Error Analysis

Fig. 4 shows the confusion matrices. For all three models, the dominant error mode is the English–French confusion (e.g., 16 French→English and 18 English→French errors for XGBoost), which is plausible given the phonetic and prosodic proximity of the two languages within the corpus. Arabic is the easiest class throughout, reaching 97% recall with the proposed model. The time-series CNN reduces the total number of misclassifications from 66 (XGBoost) and 81 (summary CNN) to 38, with the largest relative improvement precisely on the difficult English–French pair, suggesting that temporal-spectral patterns carry the discriminative information that aggregate statistics lose.

C. Training Behavior

Fig. 5 presents the epoch–loss and epoch–accuracy curves of both neural networks. Early stopping with best-weight restoration prevents overfitting; the validation curves track the training curves closely, indicating that the dropout and batch-normalization regularization is adequate for the dataset size.

D. Interactive Demonstration

The complete system is deployed as a Gradio web application in which the user records or uploads an audio clip and selects one of the three models; the clip is then passed through exactly the preprocessing chain used during training of the selected model, and class probabilities are displayed in real time.

V. CONCLUSION

We presented a controlled three-tier study of spoken-language identification on Mozilla Common Voice data for Arabic, English, and French. By extracting a summary-statistics vector and a time-series MFCC representation from the same recordings, and by keeping the convolutional backbone, data split, and training protocol fixed, we isolated the effect of the input representation on classification performance. The results are unambiguous: a 1D-CNN on summary statistics does not beat a well-tuned XGBoost baseline (86.50%

vs. 89.00%), whereas the same network on the time-series representation reaches 93.67% accuracy and nearly halves the error count. The decisive factor for this task is therefore not the adoption of deep learning per se but the preservation of temporal structure in the input. Future work includes audio data augmentation (additive noise, pitch shifting), recurrent and attention-based architectures on top of the convolutional front-end, k -fold cross-validation for tighter confidence estimates, and scaling the study to a larger number of languages and accents within the same language.

REFERENCES

- [1] S. B. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 28, no. 4, pp. 357–366, 1980.
- [2] N. Dehak, P. J. Kenny, R. Dehak, P. Dumouchel, and P. Ouellet, “Front-end factor analysis for speaker verification,” *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 19, no. 4, pp. 788–798, 2011.
- [3] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 785–794.
- [4] D. Snyder, D. Garcia-Romero, G. Sell, D. Povey, and S. Khudanpur, “X-vectors: Robust DNN embeddings for speaker recognition,” in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP)*, 2018, pp. 5329–5333.
- [5] R. Ardila *et al.*, “Common Voice: A massively-multilingual speech corpus,” in *Proc. 12th Lang. Resources Eval. Conf. (LREC)*, 2020, pp. 4218–4222.
- [6] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
- [7] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 448–456.
- [8] M. Lin, Q. Chen, and S. Yan, “Network in network,” in *Proc. 2nd Int. Conf. Learn. Representations (ICLR)*, 2014.
- [9] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [10] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. 3rd Int. Conf. Learn. Representations (ICLR)*, 2015.
- [11] L. Prechelt, “Early stopping—but when?” in *Neural Networks: Tricks of the Trade*. Springer, 1998, pp. 55–69.
- [12] M. Abadi *et al.*, “TensorFlow: A system for large-scale machine learning,” in *Proc. 12th USENIX Symp. Operating Syst. Design Implementation (OSDI)*, 2016, pp. 265–283.
- [13] B. McFee *et al.*, “librosa: Audio and music signal analysis in Python,” in *Proc. 14th Python Sci. Conf. (SciPy)*, 2015, pp. 18–25.

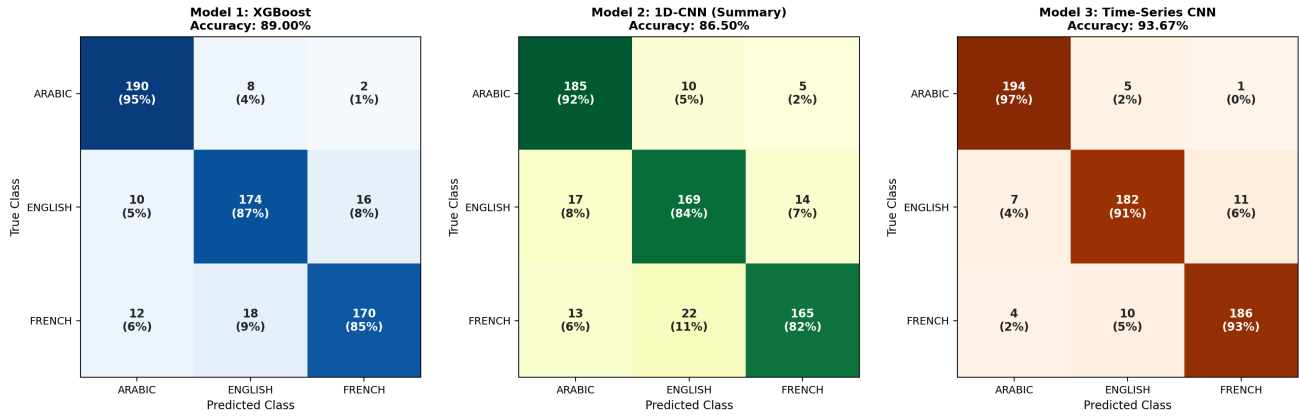


Fig. 4. Confusion matrices of the three models on the shared test set (raw counts; row-normalized percentages in parentheses). The proposed model nearly halves the error count of the baseline, with the largest gains on the English–French pair.

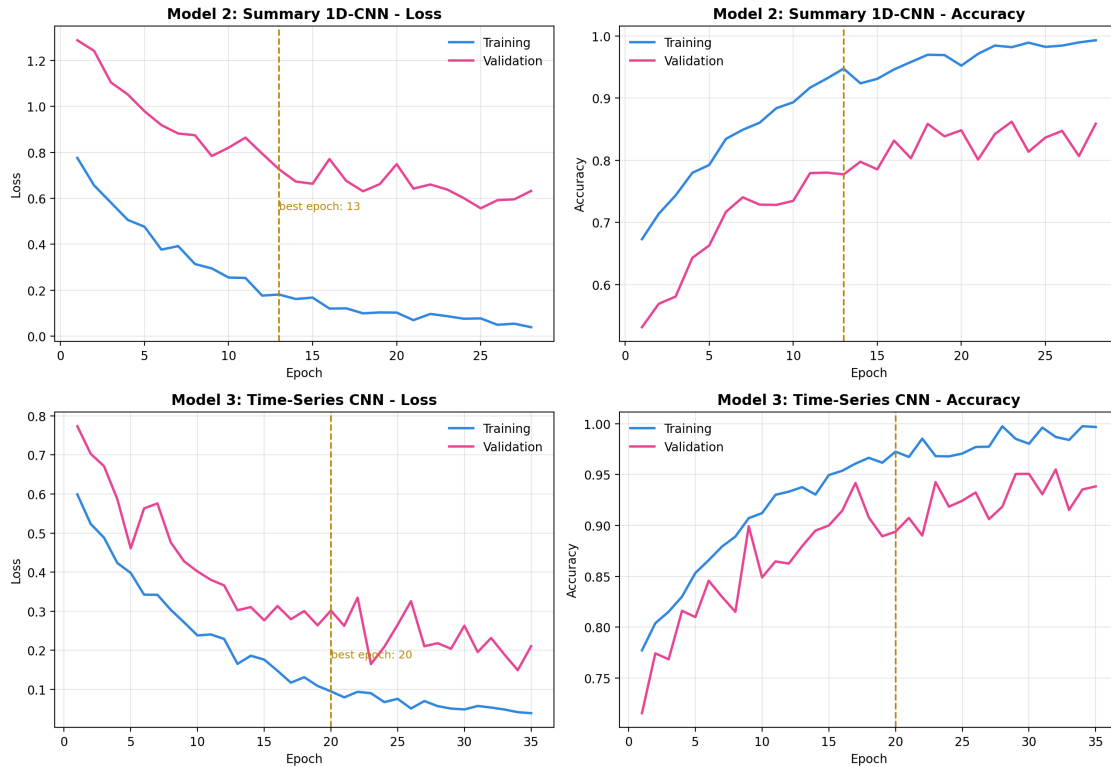


Fig. 5. Training and validation loss/accuracy curves for Model 2 (top) and Model 3 (bottom). Dashed vertical lines mark the best epoch selected by early stopping.